VINUNIVERSITY

Data Foundations

AICB-P1 · Ngày 7 · Knowledge Base

M.Sc Trần Minh Tú
Technical Director – AI Platform @ MSB

Phase 1 · Day 7 · 2026

1. Sự cần thiết của Data Foundation
2. Tổng quan về xử lý tri thức và phục vụ tri thức
3. Phân loại dữ liệu
4. Các điểm lưu ý khi làm việc với dữ liệu
5. Chunking
6. Embedding
7. Vector Store
8. Retrieval cơ bản
9. Lab Day 07 + Deliveriable

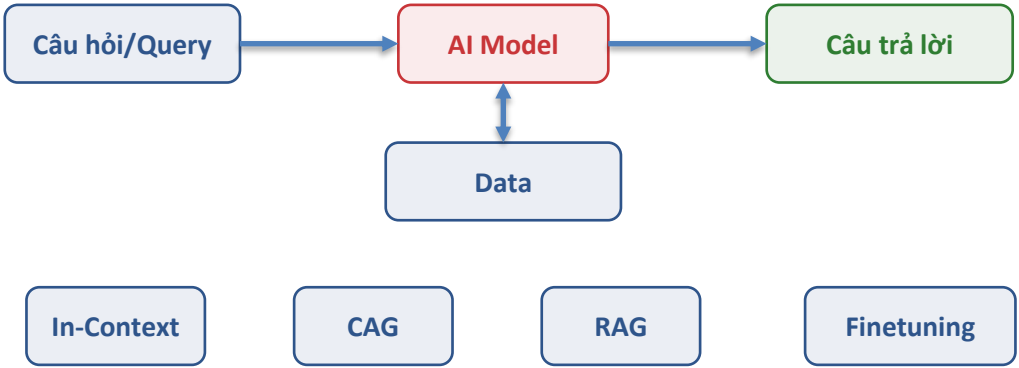
Sự cần thiết của Knowledge Base

01

Khi LLM/VLM biết câu trả lời



Khi mô hình không có sẵn câu trả lời



Các phương pháp đưa dữ liệu cho Agent

VINUNIVERSITY

In Context

Dán thẳng nội dung vào prompt:

"Dựa vào tài liệu sau: [toàn bộ policy] — Trả lời: "

Không có bước embed, không có DB.

+ Truy xuất nhanh, triển khai đơn giản

- Context window của mô hình có hạn, context ngắn và gần như không thay đổi

CAG

Sử dụng KV Cache:

Không có bước embed, không có DB.

+ Độ trễ thấp

+ Triển khai đơn giản

- Chỉ phục vụ được khi context ở mức vừa phải do dùng in-memory để chứa KV-Cache, ít thay đổi

RAG

Sử dụng Database để lưu và tìm kiếm

+ Có thể chứa lượng context rất lớn

+ Có thể dễ dàng cập nhật tri thức

- Cần pipeline xử lý phức tạp

- Độ trễ cao, độ chính xác chưa tuyệt đối

Finetuning

Train lại trọng số model:

5,000 cặp Q–A training job

model mới "thuộc" cách trả lời. Kiến thức nằm **trong** model, không nằm ở prompt.

+ Học được **giọng điệu**, format riêng

- Tốn thời gian làm Dataset

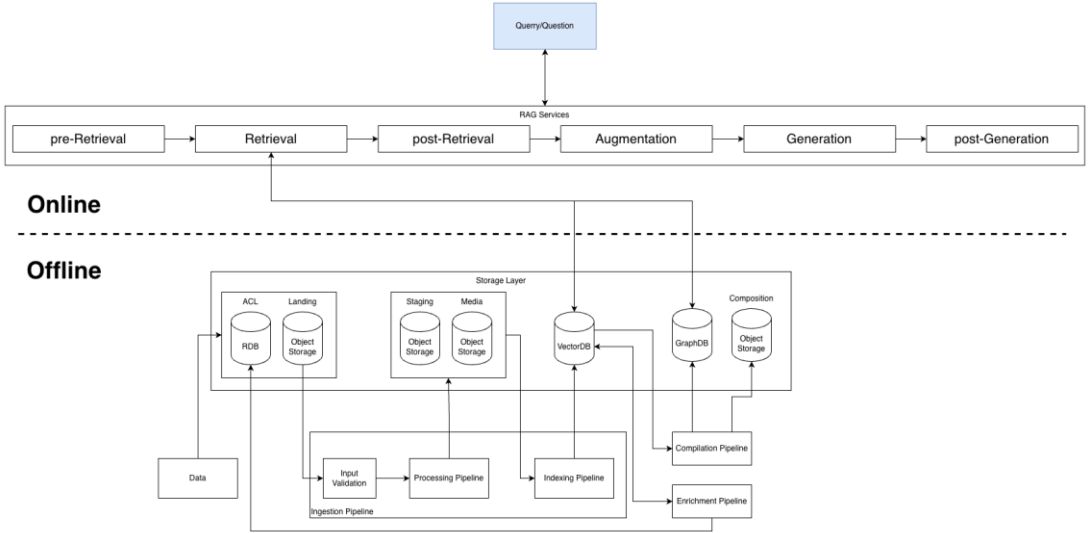
- Tốn GPU, và nhạy cảm với sự thay đổi

Lưu ý

Lưu ý: Hiểu nhầm phổ biến là dùng **finetuning để nạp kiến thức mới**. Finetuning dạy model *cách nói*; RAG cung cấp *điều cần nói*.

Fact: Finetune sẽ tốn chi phí và thời gian, nếu dữ liệu thường xuyên thay đổi RAG là sự lựa chọn hợp lý.

Tổng quan về tri thức và phục vụ tri thức



Các loại Data cung cấp Context cho Agent

VINUNIVERSITY

Knowledge Data
Tài liệu, policy, SOP, FAQ, manual, hợp đồng, bài viết nội bộ.

Operational Data
Database, trạng thái đơn hàng, ticket, CRM records, logs, transactions.

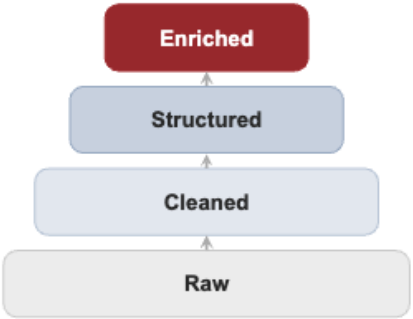
Contextual Data
Session history, user profile, preferences, recent actions, channel context.

RAG Engine

Query/MCP

Context Inject
- Short Term Memory
- Long Term Memory

Lưu ý khi làm việc với dữ liệu: Data Quality Pyramid



Tầng	Ví dụ cụ thể
Raw	PDF scan lệch, OCR ra “`đổi trả`” thành “`d0i trả`”, HTML chứa tag rác
Cleaned	“đổi trả”, bỏ header/footer, chuẩn hóa Unicode
Structured	chunk theo heading, gắn source `refund-v3.pdf`
Enriched	tag `category:support`, `access:public`, `quality:verified`

Lưu ý

Sai lầm phổ biến: index ngay dữ liệu **raw** rồi kỳ vọng retrieval sẽ tự chữa mọi vấn đề.

Lưu ý khi làm việc với dữ liệu: Data Ownership Và Governance

VINUNIVERSITY

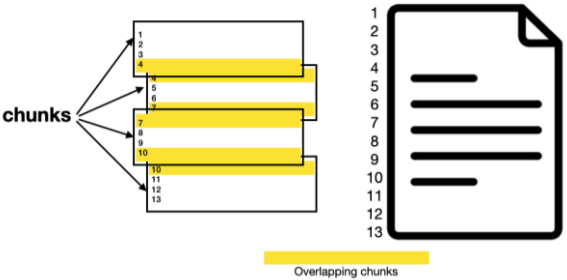
- ☑ **Ai sở hữu dữ liệu?** team nào chịu trách nhiệm cập nhật?
- ☑ **Ai được phép truy cập?** dữ liệu nào cần ACL, dữ liệu nào public nội bộ?
- ☑ **Data freshness** bao lâu phải re-index?
- ☑ **PII / sensitive fields** có cần mask trước khi embed không?
- ☐ **Không index mọi thứ theo kiểu "cứ nạp hết vào vector DB đã".**
- Ví dụ: FAQ do team CS sở hữu, public nội bộ, re-index mỗi tuần, không chứa PII.

Chunking

Chunk là đơn vị nhỏ nhất mà agent thật sự truy xuất; cắt sai ở đây thì mọi bước sau đều trả giá

03

Chunking Là Gì?



- Embedding model có giới hạn token input
- Chunk nhỏ retrieval chính xác hơn, inject ít nhiễu hơn
- 1 file dài 1 vector: chỉ tìm được file, không tìm được đoạn liên quan
- **Đưa quá nhiều context không liên quan có thể làm giảm chất lượng câu trả lời** — model bị phân tán, tốn thêm token và latency

Chiến Lược Chunking (1/3) — Nhóm Nền Tảng

Strategy	Cách cắt	Mạnh	Yếu
Fixed-size	Cắt mỗi N token, overlap 10–15%	Đơn giản, chunk đều, dễ kiểm soát cost	Cắt giữa câu, giữa bảng
Recursive	Thử tách theo n n n câu, đến khi vừa size	Tôn trọng ranh giới tự nhiên; mặc định tốt	Vẫn cắt nếu 1 đoạn quá dài
Heading / structural	Cắt theo heading Markdown (#, ##) hoặc theo section	Chunk trọn ý, metadata section rõ	Cần tài liệu có cấu trúc tốt

*Ba strategy **nền tảng** — lab yêu cầu implement cả ba rồi so sánh trên cùng một tập query, để thấy chunking là quyết định thiết kế, không phải tham số mặc định.*

Chiến Lược Chunking (2/3) — Nhóm Ngữ Nghĩa

Strategy	Cách cắt	Mạnh	Yếu
Semantic	Embed từng câu, cắt tại chỗ cosine giữa 2 câu liền kề **tụt mạnh** (topic shift)	Ranh giới đúng theo ý, không phụ thuộc heading	Tốn embedding lúc ingest; cần tune ngưỡng
Sentence-window	Index **1 câu**, nhưng inject câu đó k câu xung quanh	Retrieve rất chính xác, inject vẫn đủ ngữ cảnh	Số record lớn; cần lưu vị trí câu
Proposition / atomic	Dùng LLM viết lại đoạn thành các **mệnh đề độc lập**, mỗi mệnh đề 1 chunk	Mỗi chunk tự đủ nghĩa, ít nhiễu nhất	Đắt nhất; LLM có thể bóp méo nội dung

*Nhóm này **tôn trọng ngữ nghĩa** thay vì đếm token. Đắt hơn khi ingest (cần embed hoặc gọi LLM), nhưng thường tăng retrieval precision rõ rệt trên tài liệu dài, ít cấu trúc.*

Chiến Lược Chunking (3/3) — Nhóm Chuyên Biệt

Strategy	Cách cắt	Mạnh	Yếu
Parent-child	Index chunk **nhỏ** để search, inject chunk **cha** lớn hơn	Cân bằng precision và ngữ cảnh; rất phổ biến trong production	Cần lưu quan hệ cha–con
Hierarchical	Nhiều tầng: doc summary section paragraph, search từ trên xuống	Tốt cho corpus rất lớn, hỗ trợ câu hỏi tổng hợp	Pipeline phức tạp, phải sinh summary
Code-aware (AST)	Cắt theo hàm / class bằng AST parser	Không bao giờ cắt giữa hàm; giữ import, signature	Cần parser theo từng ngôn ngữ
Table / layout-aware	Giữ nguyên bảng, kèm lại header dòng cho mỗi phần bảng	Bảng không bị mất nghĩa khi cắt	Cần layout parser cho PDF

Quality of Chunking: Quá To Hay Quá Nhỏ Đều Trả Giá

VINUNIVERSITY

Chunk quá to

Dính nhiều chủ đề vào cùng một vector, retrieve trúng nhưng inject rất nhiễu.

> 1000 tokens

Chunk hợp lý

Một ý / một section trọn vẹn, có overlap 10–15% giữa các chunk.

200–500 tokens

Chunk quá nhỏ

Mất ngữ cảnh, retrieve nhiều mảnh rời rạc, khó tổng hợp.

< 50 tokens

Lưu ý

Rule of thumb: bắt đầu đơn giản với chunk theo section / heading, rồi tối ưu sau bằng eval thay vì đoán cảm tính.

Question: Có thật sự cần overlap không ?

Ví Dụ: Cùng Một FAQ, Ba Cách Chunk

VINUNIVERSITY

Chunk quá to

Q1: Hoàn tiền mất bao lâu? A1: 7 ngày.

Q2: Đổi size được không? A2: Trong 30 ngày.

Q3: Ship quốc tế? A3: Không.

Query "đổi size" retrieve cả 3 LLM bị nhiễu bởi Q1, Q3.

Chunk hợp lý

Q2: Đổi size được không? A2: Được, trong vòng 30 ngày kể từ ngày mua.

Query "đổi size" retrieve đúng Q2 LLM trả lời chính xác.

Chunk quá nhỏ

"Trong 30 ngày."

Query "đổi size" retrieve "Trong 30 ngày" thiếu ngữ cảnh, LLM không biết 30 ngày cho việc gì.

Quality of Chunking – Cần Overlap

- **Khi KHÔNG dùng Overlap:**
- Chunk 1: *"Công ty A đạt doanh thu 100 tỷ vào năm 2025."*
- Chunk 2: *"Mức tăng trưởng này là nhờ vào việc mở rộng thị trường xuất khẩu."*

→**Vấn đề:** Nếu người dùng hỏi *"Vì sao doanh thu năm 2025 của công ty A tăng?"*, Chunk 2 chứa câu trả lời nhưng không hề biết *"Mức tăng trưởng này"* là của ai hay năm nào.

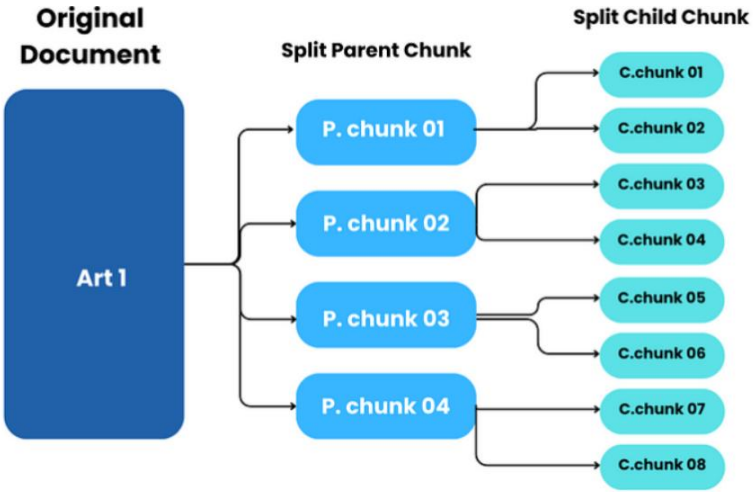
- **Khi CÓ Overlap (~30%):**
- Chunk 1: *"Công ty A đạt doanh thu 100 tỷ vào năm 2025."*
- Chunk 2: *"...doanh thu 100 tỷ vào năm 2025. Mức tăng trưởng này là nhờ vào việc mở rộng thị trường xuất khẩu."*

→**Kết quả:** Chunk 2 giữ trọn vẹn ngữ cảnh và giúp hệ thống truy vết chính xác nguyên nhân tăng trưởng.

→**Nhược điểm:** Tốn thêm dung lượng lưu trữ cho phần overlap

Quality of Chunking – Không cần Overlap

- Văn bản có cấu trúc ngữ nghĩa rõ ràng kết hợp với phương pháp chunking hợp lý như Sematic Chunking/Agentic Chunking, v.v
- Chunking phân cấp Parent & Child, hoặc theo AST nếu văn bản đã có phân cấp
- Dữ liệu có cấu trúc, bản ghi độc lập



Chọn Chunking Nào? — Cây Quyết Định Thực Dụng

- Tài liệu có heading / Markdown tốt **Heading / structural** (bắt đầu ở đây)
- Không có cấu trúc, văn bản chảy dài **Recursive**, rồi thử **Semantic**
- Retrieval trúng chunk nhưng LLM *thiếu ngữ cảnh* **Parent-child** hoặc **Sentence-window**
- Retrieval trúng chunk nhưng LLM *bị nhiễu* chunk nhỏ hơn, hoặc **Proposition**
- Corpus rất lớn, có câu hỏi tổng hợp **Hierarchical**
- Source code **Code-aware (AST)**; PDF nhiều bảng **Table-aware**

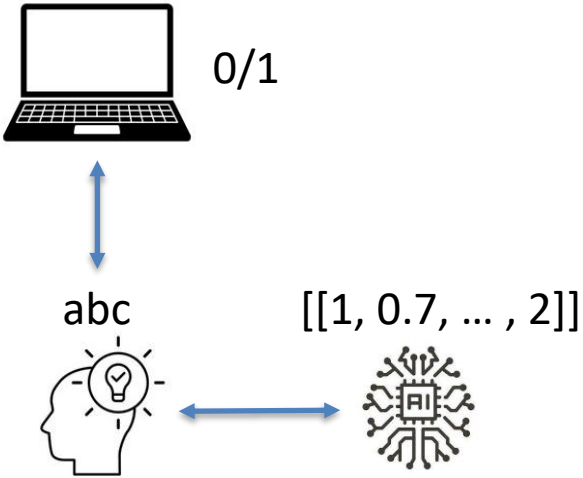
https://reference.langchain.com/python/langchain-text-splitters

Embeddings

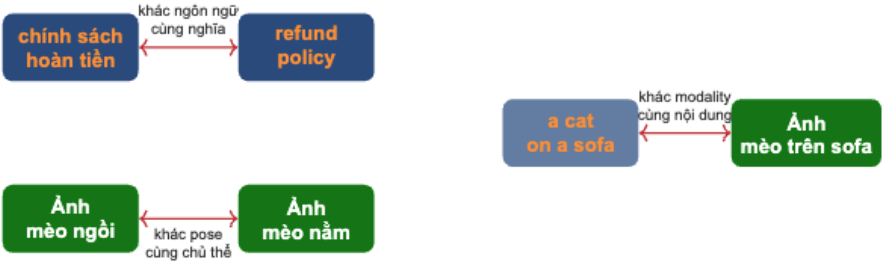
Embedding biến ngôn ngữ thành không gian toán học để máy có thể so sánh nghĩa thay vì chỉ so chữ

04

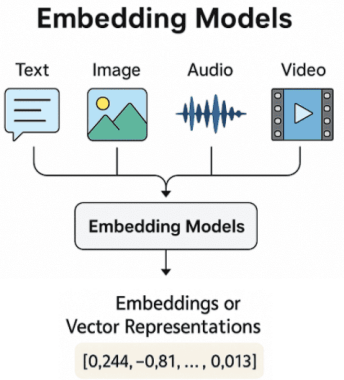
Tại sao lại cần Embedding - Bài Toán “Tương đồng”

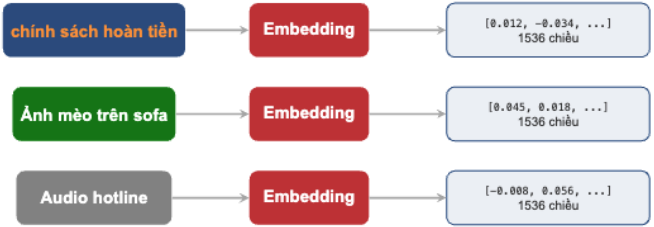


Tại sao lại cần Embedding - Bài Toán "Tương đồng"



> Con người nhìn là biết "giống nhau". Nhưng máy cần biến text, ảnh, audio thành **số** trong cùng không gian — rồi mới đo khoảng cách được.





Cùng không gian

Định nghĩa

Embedding model là hàm biến dữ liệu thô (text, ảnh, audio) thành vector số **cùng kích thước**. Sau đó, đo khoảng cách giữa các vector = đo "độ giống nhau" về nghĩa.

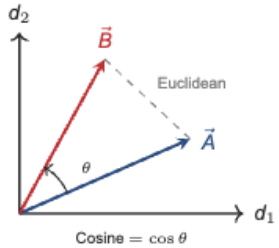
Embedding Là Gì?



Minh họa: câu có nghĩa gần nhau sẽ ở gần nhau hơn trong vector space.

- Text được biến thành vector nhiều chiều.
- Từ đó, máy có thể đo **độ gần về nghĩa**.
- Đây là nền tảng cho semantic search, clustering, dedup, recommendation.

Đo "Gần" Như Thế Nào? Cosine vs Euclidean



Cosine similarity — đo *hướng*

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

Mẫu số chia cho hai độ dài → **norm bị triệt tiêu**. 1 = cùng hướng, 0 = vuông góc, -1 = ngược.

Euclidean distance — đo *độ lớn*

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$$

Khoảng cách "đường chim bay"; 0 = trùng nhau. **Có** tính norm → vector dài/ngắn cho kết quả khác nhau.

1536 chiều: công thức y hệt, chỉ nhiều i hơn — không cần tự code, nhưng cần hiểu score 0.87 vs 0.31 nghĩa là gì.

Khái Quát: Khi Nào Cosine, Khi Nào Euclidean?

VINUNIVERSITY

Câu hỏi gốc: *"độ lớn của vector có mang thông tin cần so sánh hay không?"*

Dùng COSINE khi…

Chỉ quan tâm **nội dung / hướng**, không quan tâm độ lớn — dữ liệu dài ngắn rất khác nhau nhưng cùng chủ đề. Vector thưa, nhiều chiều: text embedding, TF-IDF, user preference.

Ví dụ: "hoàn tiền" (5 từ) vs cả trang policy hoàn tiền → cosine vẫn thấy giống nhau.

Dùng EUCLIDEAN khi…

Độ lớn là thông tin thật, không phải nhiễu, và các chiều cùng đơn vị / cùng scale. Cần khoảng cách tuyệt đối: toạ độ, cảm biến, image feature, clustering (k-means).

Ví dụ: hai điểm GPS — 2 km và 200 km khác nhau thật, không được chuẩn hóa mất.

Khi vector đã **normalize** về độ dài 1, cosine và Euclidean **xếp hạng giống nhau**. Phần lớn embedding model hiện đại đều trả vector đã normalize → mặc định cứ dùng **cosine** cho text.

Ví Dụ Trực Quan: Score Nghĩa Là Gì?

VINUNIVERSITY

Câu A	Câu B	Cosine	Nghĩa
“Chính sách hoàn tiền”	“Quy định đổi trả”	0.87	Rất gần nghĩa
“Chính sách hoàn tiền”	“Cách giao hàng nhanh”	0.52	Liên quan
“Chính sách hoàn tiền”	“Thời tiết hôm nay”	0.31	Không liên quan
“Chính sách hoàn tiền”	“Refund policy”	0.82	Cross-lingual!

*Embedding model hiểu nghĩa **cross-lingual**: “hoàn tiền” và “refund” gần nhau dù khác ngôn ngữ. Đây là sức mạnh so với keyword search.*

Embedding Dùng Để Làm Gì Trong AI Product?

Use case	Cách hoạt động	Ví dụ thực tế
Semantic search	Embed query + embed tài liệu, tìm cosine cao nhất	User hỏi "trả hàng" tìm được chunk "chính sách đổi trả" dù không trùng từ
Clustering	Gom các vector gần nhau thành nhóm	10,000 ticket CS tự phân thành 15 chủ đề
Dedup	So cosine giữa các cặp, đánh dấu cặp > threshold	Phát hiện 2 bài FAQ nội dung gần giống, gộp lại
Recommendation	Embed user intent + embed item, rank theo cosine	User vừa đọc "React hooks" gợi ý "useState patterns"

Embedding không tự tạo ra "sự thật". Nếu text nguồn sai, thiếu, hoặc chunk xấu thì retrieval vẫn tệ — garbage in, garbage out.

Chọn Embedding Model: Trade-off Thực Tế

Model	Dim	MTEB Avg	Khi nào dùng
`text-embedding-3-small`	1536	62.3	Demo, PoC, lab (rẻ, nhanh)
`text-embedding-3-large`	3072	64.6	Production cần chất lượng
Open-source (e5, BGE, Qwen Embedding, Jina Embedding, v.v)	768–4096	61–66	Self-host, data nhạy cảm, free

Lưu ý

Có hàng trăm embedding model (Cohere, Voyage, Gemini, Jina). Xem bảng xếp hạng đầy đủ: MTEB Leaderboard trên Hugging Face.

Vector Store

Vector store không chỉ lưu vector; nó lưu cả nguồn, metadata, và khả năng lọc để retrieval có ngữ cảnh đúng

05

Vector Store Lưu Những Gì?

VINUNIVERSITY

Mỗi record gồm 4 thành phần

- ID
- Original chunk (text gốc)
- Embedding vector
- Metadata (key-value)

- Search results (top-k)

Output khi query, không phải thứ được lưu

Ví dụ 1 record thật

Trường	Giá trị
ID	`policy-returns-001`
Chunk	"Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem."
Vector	`[0.012, -0.034, ...]` (1536 số)
Metadata	`source: refund-v3.pdf, category: support, updated: 2026-03-01, access: public-internal`

Vector dùng để tìm (semantic search). **Chunk** dùng để inject vào prompt cho LLM. Hai thứ khác nhau, lưu cùng nhau.

Metadata Quan Trọng Không Kém Similarity

Trường metadata	Ví dụ	Tác dụng
Source / file name	`refund-policy-v3.pdf`	truy vết và hiển thị nguồn
Category	`finance`, `support`, `legal`	lọc đúng domain trước khi search
Time / freshness	`2026-03-01`	tránh dùng nội dung cũ
Access level	`public-internal`, `restricted`	giới hạn truy cập
Section / chunk id	`returns.section.4`	debug và cite chính xác hơn

*Retrieval tốt cần **semantic similarity** kết hợp **metadata filtering**.*

Vector Store Phổ Biến (1/2) — Open Source

Tên	Dạng	Điểm mạnh	Phù hợp khi
Chroma	Embedded / server (Python)	Cài 1 dòng `pip`, API cực gọn, chạy in-memory	Học, prototype, PoC nhỏ — **dùng trong lab hôm nay**
FAISS	Library (Meta)	Nhanh nhất cho ANN search thuần, nhiều loại index	Cần tốc độ, tự quản lý; **không** phải DB (không metadata filter, không persistence sẵn)
Qdrant	Server (Rust)	Filter metadata mạnh, hiệu năng tốt, API rõ ràng	Production tự host, cần filter phức tạp
Milvus	Distributed (LF AI)	Scale tỉ vector, GPU index, cloud-native	Corpus rất lớn, có team hạ tầng
Weaviate	Server (Go)	Hybrid search sẵn, schema + module vectorizer	Cần hybrid keyword + vector
pgvector	Extension cho PostgreSQL	Dùng luôn DB đang có, JOIN với data quan hệ, ACID	Đã có Postgres và corpus vừa phải — lựa chọn "nhàm chán" nhưng rất hiệu quả

Vector Store Phổ Biến (2/2) — Commercial & Managed

Tên	Dạng	Điểm mạnh	Lưu ý
Pinecone	SaaS thuần	Managed hoàn toàn, serverless, độ tin cậy cao	Vendor lock-in; chi phí tăng theo số vector
Elasticsearch / OpenSearch	Search engine + vector	Hybrid BM25 + vector rất mạnh, hệ sinh thái lớn	Nặng để vận hành nếu chỉ cần vector
Azure AI Search	Managed (Microsoft)	Tích hợp sẵn pipeline ingest, OCR, hybrid	Gắn với hệ sinh thái Azure
Vertex AI Vector Search	Managed (Google)	Scale rất lớn, độ trễ thấp	Cấu hình index phức tạp hơn
MongoDB Atlas / Redis	DB có thêm vector index	Dùng luôn DB sẵn có, giảm số hệ thống	Tính năng vector ít sâu hơn DB chuyên dụng
File Search API (OpenAI, Anthropic)	Retrieval as a service	Không cần chunk / embed / store — gọi API là xong	Ít kiểm soát chunking, metadata, và cost path

*Nhóm **open source**: tự host, không phí license, kiểm soát dữ liệu — nhưng phải tự lo scaling, backup, monitoring. Hầu hết đều có bản managed cloud của chính hãng.*
*Nhóm **commercial / managed**: trả tiền để khỏi lo hạ tầng. Đổi lại là chi phí theo scale và phụ thuộc vendor — cần tính **cost path** trước khi cam kết.*

Chọn Vector Store: Điều Gì Thật Sự Quyết Định?

VINUNIVERSITY

Tiêu chí nên hỏi trước

- Bao nhiêu vector? (10^4 vs 10^9 là hai bài toán khác nhau)
- Có cần **metadata filter** phức tạp không?
- Có cần **hybrid search** (keyword + vector)?
- Dữ liệu có ràng buộc **compliance** / on-prem?
- Team có ai vận hành hạ tầng không?

Đường đi thực dụng

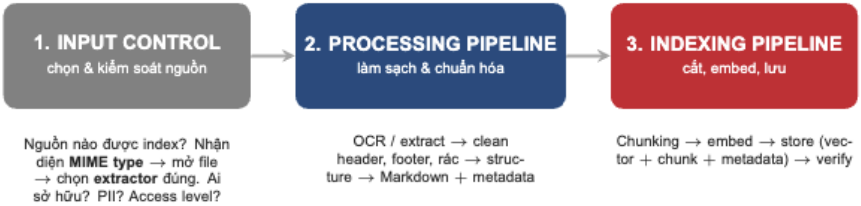
- Học / prototype → **Chroma**
- Đã có Postgres → **pgvector**
- Production tự host → **Qdrant** / **Weaviate**
- Không muốn lo hạ tầng → **Pinecone** / managed
- Cần hybrid + đã có ELK → **Elasticsearch**
- Tỉ vector → **Milvus** / Vertex

Lưu ý: Đổi vector store là việc **tương đối dễ** — interface đều là add / query / filter. Đổi **embedding model** thì phải re-index toàn bộ. Vì vậy đừng dành quá nhiều thời gian chọn store ở giai đoạn đầu.

Ingestion Pipeline

Bao gồm 3 bước nhỏ: Input Validate + Processing Pipeline + Indexing PIpeline

06



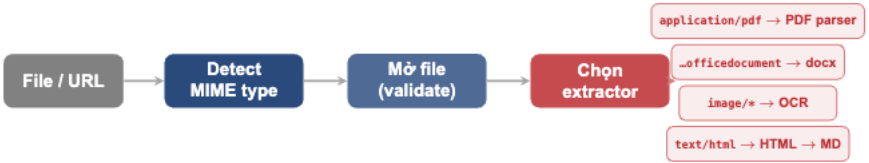
Lưu ý: Lỗi retrieval thường **không** sinh ra ở khối 3. Nó sinh ra ở khối 1 (index sai nguồn) hoặc khối 2 (bỏ bước clean) — nhưng chỉ **lộ ra** khi query. Debug thì đi ngược từ 3 về 1.

Target Output – Select a format

Format	Token hiệu quả	Giữ cấu trúc	Khi nào dùng
Markdown	★★★★★	Heading, list, table, bold	Mặc định trong nhiều trường hợp
HTML (clean)	★★★	Bảng lồng, colspan, layout phức tạp	Khi MD mất thông tin cấu trúc
Plain Text	★★★★	Không có cấu trúc	Email, chat log, transcript
JSON / YAML	★★★	Key-value, nested objects	Structured output, function calling

*Nên chọn **Markdown**. So với HTML cùng nội dung, tiết kiệm ∼30–50% token vì không có closing tags, attributes, class names. LLM được train trên lượng lớn MD nên hiểu rất tự nhiên.*

Ingestion Pipeline – Input Control (Input Validation)

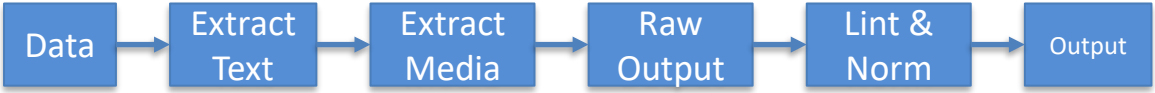


Đừng tin phần mở rộng file

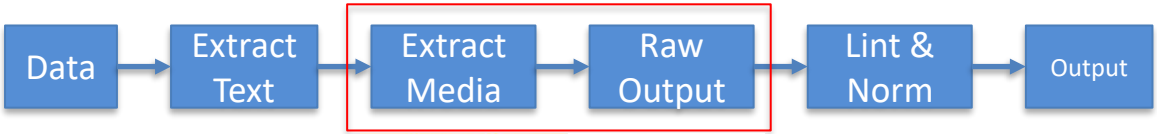
`.pdf` có thể là scan ảnh; `.txt` có thể là JSON; `.doc` có thể là RTF. Kiểm tra **magic bytes** / MIME thật, không chỉ đọc đuôi file.

Fail nhanh, ghi log rõ

Không có extractor → **bỏ qua và log**, đừng index rác. Ghi lại `mime`, `extractor`, `status` vào metadata để debug về sau.



- OCR
- File Decoder




```
![cute pig](file:///Users/apollox/Downloads/
001_500_rb-image-2672583.png)

![another cute pig](assets/
this%20is%20a%20super%20cute%20pig)

![cutest pig](x-devonthink-item://
B0E0170A-40F9-4E71-9185-FD8818EF6AAA)
```



cute pig



another cute pig



cutest pig

Ingestion Pipeline – Indexing Pipeline



- Chunking
  Type

- Model

- Dims

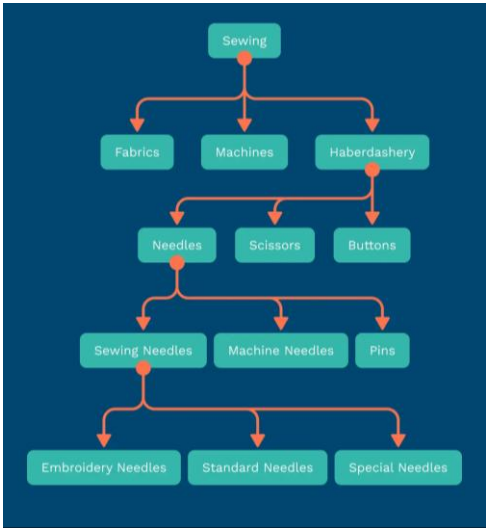
- Techstack
- Schema
- Metadata

Enrichment Pipeline

07

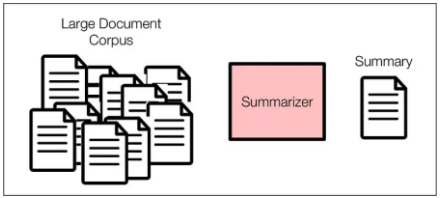
Context có thể được làm giàu để cung cấp chính xác hơn hoặc thêm thông tin cho AI

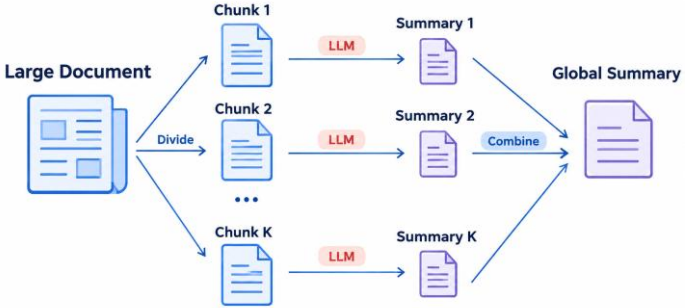
Basic Enrichment – Classification (Auto Tagging)

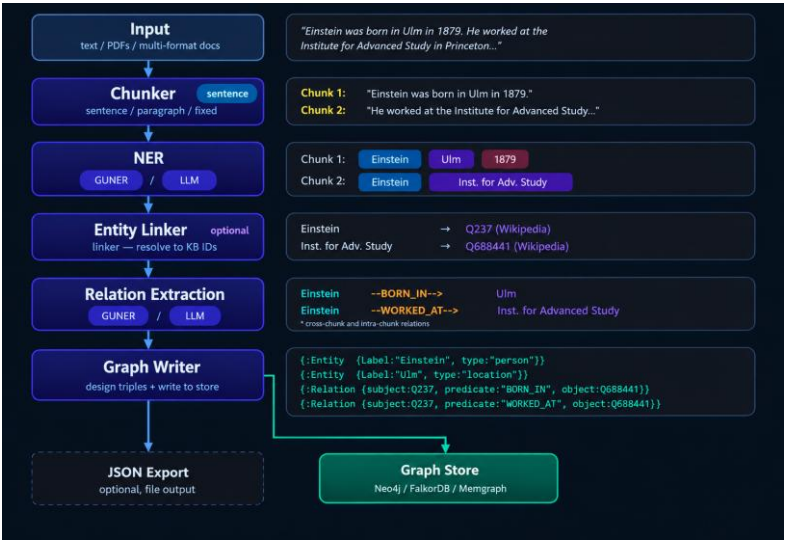




Basic Enrichment – Summarization







Suggestion:

- LLM Agent + Skill
- Graphrag-toolkit

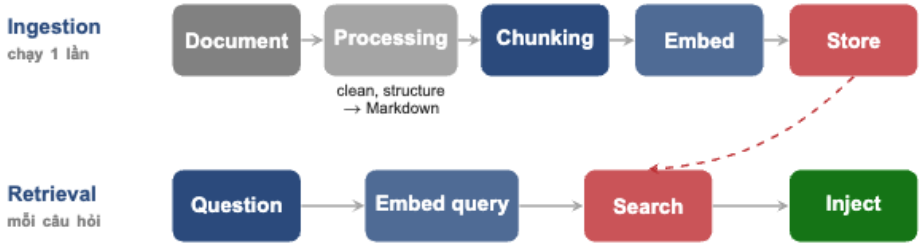
Introduce to Compilation Pipeline

Một knowledge base với nhiều văn bản tương đồng, mâu thuẫn có thể khiến hệ thống của bạn hoạt động kém hơn

Kết Nối Agent Với Data

Retrieval pipeline là chiếc cầu nối giữa dữ liệu riêng và hành vi của agent

09



Lưu ý: Ingestion chạy **offline** khi dữ liệu thay đổi. Retrieval chạy **online** mỗi khi user hỏi. Hai pha này tách biệt nhau.

Tham số	Ý nghĩa	Ví dụ
Top-k	Lấy bao nhiêu chunk gần nhất	`n_results=3`: đủ cho câu hỏi đơn. `n_results=10`: khi cần tổng hợp nhiều nguồn
Score threshold	Bỏ chunk có cosine thấp hơn ngưỡng	`0.7`: chỉ giữ chunk thật sự liên quan. `0.4`: chấp nhận liên quan nhẹ
Attribute filter	Lọc theo metadata trước khi search	`where={"category": "support"}`: chỉ tìm trong tài liệu CS

Filter theo category trước → semantic search → lấy top-3 có score > 0.6. Retrieval tốt = chunk đúng + metadata đủ + filter hợp lý — không phải đổi model.

Hands-on 7

Mục tiêu của lab là nối được dữ liệu riêng vào hệ thống AI theo pipeline tối thiểu nhưng đúng bản chất

Phase 1 — Cá nhân

Hoàn thành TODO trong 3 file:

1. `src/chunking.py` — 3 chunking strategies + cosine similarity
2. `src/store.py` — `EmbeddingStore` (5 methods)
3. `src/agent.py` — `KnowledgeBaseAgent` (RAG)

`Document` và `FixedSizeChunker` đã implement sẵn làm ví dụ. Verify: `pytest tests/ -v`

Phase 2 — Nhóm: So sánh Strategy

1. Nhóm chọn domain + thu thập 5–10 docs
2. Thống nhất 5 benchmark queries + gold answers
3. Mỗi người thử strategy riêng (chunking, metadata)
4. Chạy cùng queries, so sánh kết quả
5. Demo + thảo luận liên nhóm

Lưu ý: Cùng code, cùng model — khác data strategy sẽ cho kết quả rất khác. **Data quality > model selection.**

Deliverable Và Assessment

Phần	Điểm
Cá nhân (code + phân tích)	60
Nhóm (strategy + so sánh)	40
Tổng	**100**

- **Retrieval Precision** — top-3 có đúng không?
- **Chunk Coherence** — chunk giữ được ý trọn vẹn?
- **Metadata Utility** — filter có giúp tăng chính xác?
- **Grounding Quality** — answer dựa trên context?
- **Data Strategy Impact** — strategy phù hợp domain?

Checklist Trước Khi Nộp

- ☑ **Tests pass:**pytest tests/ -v không có FAIL
- ☑ **3 chunking strategies** đều implement và so sánh được
- ☑ **EmbeddingStore** search + filter + delete hoạt động
- ☑ **KnowledgeBaseAgent** trả lời dựa trên retrieved context
- ☑ **5 benchmark queries** có gold answers và kết quả so sánh
- ☑ **Report** giải thích approach + failure cases

Key Takeaways

Day 07 đặt nền cho RAG và mọi hệ thống AI dùng tri thức riêng

11

1. In-Context, CAG, RAG, Finetuning
2. Data Type For AI
3. Data quality, Ownership and Governane
4. Ingestion Pipeline
5. Enrichment Pipeline
6. Compilation Pipeline
7. Vector-less

Tài Liệu Tham Khảo

- Stanford HAI. *The 2025 AI Index Report*. https://hai.stanford.edu/ai-index/2025-ai-index-report
- OpenAI Docs. *Embeddings Guide*. https://platform.openai.com/docs/guides/embeddings
- OpenAI Docs. *File Search*. https://platform.openai.com/docs/guides/tools-file-search
- Hugging Face. *MTEB Benchmark*. https://huggingface.co/blog/mteb
- Chroma Documentation. https://docs.trychroma.com/
- Karpukhin et al. *Dense Passage Retrieval for Open-Domain QA*. arXiv:2004.04906